Rapport Projet RL - TP2 AI Driver Unity (PPO)

Auteur: [Nom Prenom]
Date: [JJ/MM/AAAA]
Projet: Autonomous Car - PPO
Matiere: [Nom du cours]

Introduction

Ce TP2 consiste a entrainer un agent de voiture dans Unity avec ML-Agents et PPO.
Le but est simple: faire rouler la voiture sans crash.

Le rapport peut rester leger. L'important est surtout de montrer:

1. la configuration de depart,
2. les changements faits entre les experiences,
3. ce qu'on observe sur les courbes,
4. la configuration retenue a la fin.

Rappel du projet

Fichiers principaux

- `results/configuration_example.yaml` : configuration PPO.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_controller.cs` : mouvement de la voiture.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_agent.cs` : classe de base de l'agent.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_agent_template_track1.cs` : script a completer pour le TP2.

Scripts a citer dans le rapport

- `results/configuration_example.yaml` : fichier de configuration du PPO.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_controller.cs` : controle les roues, l'acceleration, le freinage et la direction.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_agent.cs` : gere l'initialisation commune de l'agent et les episodes.
- `AI_in_games_unity/Assets/Scripts/car_agents/car_agent_template_track1.cs` : classe a completer pour connecter l'agent a l'environnement.

Ce qu'il faut faire dans Unity

- Completer `_getTarget()` et `_getStart()` .
- Completer `CollectObservations()` .
- Completer les fonctions de reward.
- Completer `OnActionReceived()` .
- Regler les parametres de l'agent dans l'inspecteur.

Parametres Unity a mentionner

- Behavior Name.
- Vector Observation space size.
- Stacked Vectors.
- Discrete branches.
- Model / Inference Device / Deterministic Inference si un modele est charge.

Methodologie

Point important sur PPO

PPO est une methode policy-based. On peut simplement dire que l'algorithme apprend une politique stable grace a des mises a jour controlees.

Parametres utiles a tester

- Batch size.
- Buffer size.

- Learning rate.
- Epsilon.
- Beta.
- Lambda.
- Num epoch.
- Gamma.

Remarque sur le batch size

Le prof a surtout insiste sur le batch size.
Si le batch size augmente, l'optimisation utilise plus d'observations, donc les mises a jour sont souvent plus stables mais aussi plus lourdes.

E1 a lancer en premier

Le premier run sert de baseline.
Il faut garder la configuration par defaut, lancer l'entrainement sur la scene track1, puis commenter les courbes obtenues sans modifier trop de choses.

Convention de nommage des runs

- E1: `Experience1`
- E2: `Experience2`
- E3: `Experience3`

Exemple de lancement E1:

```
mlagents-learn results/configuration_example.yaml --run-id=Experience1 --force
```

Experiences

> Consigne: faire 3 experiences maximum, pas besoin d'aller trop loin.

Tableau rapide

Experience	Batch size	lr	Epsilon	Beta	Lambda	Num epoch	Attente principale
E1	512	3e-4	0.2	0.005	0.95	3	Baseline
E2	1024	1.5e-4	0.15	0.003	0.95	3	Reward finale plus elevee mais moins stable
E3	768	2.2e-4	0.17	0.004	0.95	3	Compromis stabilite-rapidite: apprentissage rapide + inference robuste

Structure courte pour chaque experience

Experience E1

- **Choix des parametres**: batch size 512, learning rate 3e-4, epsilon 0.2, beta 0.005, lambda 0.95, num epoch 3.
- **Ce qu'on teste**: la baseline PPO sur track1.
- **Resultats observes**:
  - Run lance avec `--run-id=Experience1` .
  - 🖼️ **[IMAGE REQUISE - E1 Reward Curve]** Capture TensorBoard de `Environment/Cumulative Reward` pour E1 seule.
  - La courbe `Environment/Cumulative Reward` monte globalement tout le long du run.
  - D'apres l'export JSON, la valeur passe d'environ `-2.57` au debut a un pic proche de `65.22` (step `545000`), puis termine vers `54.51` (step `555000`) apres un dip suivi d'une remontee.
  - 🖼️ **[IMAGE REQUISE - E1 Episode Length]** Capture TensorBoard de `Environment/Episode Length` pour E1.
  - `Environment/Episode Length` reste bruitee et souvent proche du plafond (`624`), avec quelques baisses ponctuelles (par exemple autour de `453` au step `80000`).
  - 🖼️ **[IMAGE REQUISE - E1 Losses]** Capture TensorBoard de `Losses/Policy Loss` ET `Losses/Value Loss` pour E1 ensemble.
  - `Losses/Policy Loss` oscille dans une plage assez stable, globalement entre `0.027` et `0.045` .

- `Losses/Value Loss` augmente progressivement (d'environ `0.007` au debut jusqu'a ~ `1.10` en fin de run), avec des fluctuations.
- Test en inference apres entrainement: premier essai en echec (collision mur), deuxieme essai valide en `23.26 s` pour un tour.

- **Interpretation**: la baseline apprend bien (reward en hausse nette), mais l'agent n'est pas encore regulier. La longueur d'episode souvent elevee et le premier crash en inference montrent qu'il y a encore un manque de robustesse. Le policy loss reste raisonnable, alors que le value loss qui grimpe suggere que le critic suit plus difficilement quand les retours deviennent plus eleves.
- **Duree du run**: environ `46 minutes`.
- **Critere d'arret**: arret manuel apres un dip qui a ete re-egalise, avec une courbe reward qui restait globalement ascendante.

Experience E2

- **Choix des parametres**:
  - `batch_size` : `512 -> 1024`
  - `learning_rate` : `3e-4 -> 1.5e-4`
  - `epsilon` (PPO clip): `0.2 -> 0.15`
  - `beta` (entropy): `0.005 -> 0.003`
  - `lambda` et `num_epoch` inchanges (`0.95` , `3`)
- **Ce qu'on teste**:
  - Objectif E2: garder la progression reward de E1, mais rendre la conduite plus propre et plus reguliere.
  - Cible pratique: moins de collisions en inference et un temps moyen de tour plus bas.
- **Pourquoi ces changements**:
  - `batch_size` plus grand pour lisser les gradients et limiter les comportements erratiques.
  - `learning_rate` plus bas pour eviter des mises a jour trop agressives (E1 montrait une variance notable).
  - `epsilon` plus petit pour rendre les updates PPO plus prudentes, donc plus stables.
  - `beta` legerement reduit pour diminuer le cote trop aleatoire en fin d'apprentissage et gagner en trajectoire propre.
- **Resultats observes**:
  - Run lance avec `--run-id=Experience2`.
  - 🖼️ **[IMAGE REQUISE - E2 Reward Curve]** Capture TensorBoard de `Environment/Cumulative Reward` pour E2.
  - Cumulative reward: -2.53 (debut) → 60.05 (step 705k, **plus eleve que E1 final!**)
  - 🖼️ **[IMAGE REQUISE - E2 Episode Length]** Capture TensorBoard de `Environment/Episode Length` pour E2.
  - Episode length: Similaire a E1, souvent proche du max 624 avec dips ponctuels.

- 🖼️ **[IMAGE REQUISE - E2 Losses]** Capture TensorBoard de `Losses/Policy Loss` ET `Losses/Value Loss` pour E2 (bien montrer value loss qui monte a 1.47!).
  - Policy loss: Oscille entre 0.024-0.030 (stable, comparable a E1).
  - Value loss: Monte a 1.47 (legerement plus instabile que E1 qui atteignait max 1.10-1.25).
  - Tests en inference: plusieurs crashes et comportements erratiques (agent reste bloque apres avoir rate une reward, crash au debut d'autres parcours). Un essai a complete le tour en `20.18 sec` (plus rapide que E1).
- **Interpretation**: E2 converge vers une reward finale plus elevee que E1 (+5.54), suggerant un apprentissage plus "agressif" et potentiellement plus rapide. Cependant, la value loss plus elevee indique que le critic a plus de difficulte a suivre les dynamiques de reward, ce qui se traduit par une moindre robustesse en inference: plusieurs crashes et comportements erratiques. Le temps tour reussi (20.18 sec vs 23.26 sec) montre que quand l'agent ne crash pas, il est plus rapide. Le compromis est donc: meilleure performance finale, mais moins de stabilite. Le batch size augmente + learning rate reduit ont permis une meilleure convergence, mais au prix de la robustesse.

Experience E3

- **Choix des parametres**:
  - `batch_size` : `E1=512, E2=1024 → E3=768` (compromis intermédiaire)
  - `learning_rate` : `E1=3e-4, E2=1.5e-4 → E3=2.2e-4` (plus proche de E1 pour réduire instabilité)
  - `epsilon` (PPO clip): `E1=0.2, E2=0.15 → E3=0.17` (un peu plus de flexibilité que E2)
  - `beta` (entropy): `E1=0.005, E2=0.003 → E3=0.004` (milieu pour équilibrer exploration)
  - `lambda` et `num_epoch` inchanges (`0.95` , `3`)
- **Ce qu'on teste**:
  - Objectif E3: atteindre un **compromis optimal** entre les deux approches.
  - Cible pratique: une reward finale proche de E2 (convergence rapide), MAIS avec une robustesse d'inference proche de E1 (peu de crashes).
- **Pourquoi ces changements**:
  - **Batch size 768** (au lieu de 512 ou 1024): Assez grand pour lisser les gradients comme E2, mais pas trop pour éviter l'instabilité de la value loss. Represente un juste-milieu computationnellement.
  - **Learning rate 2.2e-4** (au lieu de 3e-4 ou 1.5e-4): Compromise entre la stabilité relative de E1 (3e-4 trop haute) et la convergence rapide de E2 (1.5e-4 trop basse et instable). Une valeur plus proche de E1 devrait réduire les swings violents observés en value loss E2, tout en gardant une progression acceptable.
  - **Epsilon 0.17** (au lieu de 0.2 ou 0.15): PPO clip légèrement plus permissif que E2 (0.15 était peut-être trop restrictif), mais plus strict que E1 (0.2 laissait trop de variance). Permet une meilleure adaptation comportementale en inference.

    - **Beta 0.004** (au lieu de 0.005 ou 0.003): Entrepose l'exploration (β bas = moins d'aleatoire) et l'exploitation (β haut = plus d'aleatoire). Equilibre entre la tendance a rester bloque de E1 et l'erraticité de E2.
- **Resultats observes**:
    - Run lance avec `--run-id=Experience3`.
    - Training duration: environ `50 minutes` pour atteindre 650k steps (`max_steps: 650000` arrêt automatique).
    - 🖼️ **[IMAGE REQUISE - E3 Reward Curve]** Capture TensorBoard de `Environment/Cumulative Reward` pour E3 (montrer la progression PROPRE et FULGURANTE!).
    - Cumulative reward: -2.53 (debut) → 55.77 (step 650k)
    - 🖼️ **[IMAGE REQUISE - E3 Episode Length]** Capture TensorBoard de `Environment/Episode Length` pour E3.
    - Episode length: Similaire a E1/E2, souvent proche du max 624 avec dips ponctuels.
    - 🖼️ **[IMAGE REQUISE - E3 Losses]** Capture TensorBoard de `Losses/Policy Loss` ET `Losses/Value Loss` pour E3 (IMPORTANT: montrer value loss a 1.22, cle entre E1 et E2!).
    - Policy loss: Oscille entre 0.025-0.027 (tres stable, comparable a E1 et E2).
    - Value loss: Monte a 1.22 (entre E1 ~1.10-1.25 et E2 ~1.47) → meilleur que E2!
    - Tests en inference: agent observe plus stable, rare de rater une reward ou de crash dans les virages serre. Temps tour mesure: `20.91 sec`.
- **Interpretation**: E3 atteint le **compromis optimal** recherche: reward finale 55.77 (entre E1 54.51 et E2 60.05), value loss re-stabilisee a 1.22 (meilleure que E2), et surtout une **robustesse en inference nettement amelioree** vs E2. La progression est "propre et fulgurante" avec peu d'oscillations et un gain quasi constant, suggerant un apprentissage plus regulier. Le temps tour 20.91 sec est entre E2 (20.18s, instable) et E1 (23.26s, stable), ce qui correspond exactement au compromise desire. E3 aurait probablement gagne a tourner plus longtemps (aurai pu atteindre 57-58 en reward), mais les 50 min investies donnent deja un bon resultat.

Resultats

Figures a mettre dans le rapport

- **Figure 1 (Reward):** 🖼️ **[IMAGE REQUISE - Comparaison E1/E2/E3]** Capture TensorBoard de `Environment/Cumulative Reward` montrant les 3 runs ensemble (ou cote a cote si scrollable). Commenter la hausse de E1, l'agressivite de E2, et la progression propre de E3.

- **Figure 2 (Episode Length):** 🖼️ **[IMAGE REQUISE - Comparaison Episode Length]** Capture TensorBoard de `Environment/Episode Length` pour les 3 configs. Noter que les trois configs restent souvent au max 624, mais E3 est plus regulier.
- **Figure 3 (Policy Loss):** 🖼️ **[IMAGE REQUISE - Policy Loss E1/E2/E3]** Capture TensorBoard de `Losses/Policy Loss` pour les 3. Montrer la stabilite relative des trois (E3 entre E1 et E2).
- **Figure 4 (Value Loss - CRUCIAL):** 🖼️ **[IMAGE REQUISE - Value Loss E1/E2/E3 ENSEMBLE]** Capture TensorBoard de `Losses/Value Loss` montrant clairement E1, E2, E3 pour BIEN VOIR LA DIFFERENCE. **IMPORTANT** : bien mettre en evidence que **E2 monte a 1.47** (instable/erratique) et **E3 reste a 1.22** (meilleur compromis) vs E1 ~1.10-1.25. **C'est la courbe cle qui explique pourquoi E3 gagne!**

Images / captures a commenter (comportement & gameplay)

- **Figure A:** 🖼️ **[IMAGE REQUISE - Screenshot Unity]** Voiture au depart de la track1 scene (agent initialise correctement dans le start position, avant entrainement).
- **Figure B:** 🖼️ **[IMAGE REQUISE - Screenshot Unity]** Voiture qui atteint la cible/target (montre que le reward shaping fonctionne, donne l'intuition du but du jeu).
- **Figure C:** 🖼️ **[IMAGE REQUISE - Screenshot Unity]** (Optionnel) Voiture qui crash ou sort de la route (ex: E2 qui crash dans un virage serre, illustre la robustesse inferieure observee).
- **Figure D:** 🖼️ **[IMAGE REQUISE - Zoom TensorBoard]** Courbe reward de E1 seule (avec region annotees sur les zones bruitees early game et stabilisees late game).
- **Figure E:** 🖼️ **[IMAGE REQUISE - Zoom TensorBoard]** Courbe episode length de E1 (pour montrer le bruit inherent de ce metrique, utilise comme diagnostic de chaos vs stabilite).
- **Figure F:** 🖼️ **[IMAGE REQUISE - Comparaison Value Loss E2 vs E3]** (Optionnel mais recommande) Capture TensorBoard montrant value loss de E2 et E3 cote a cote ou superposees pour bien voir que E2 diverge (1.47) tandis que E3 reste discipline (1.22). Annote les deux courbes.

Comparaison finale

Critere	E1	E2	E3
Reward finale	54.51 (step 555k)	**60.05 (step 705k)**	**55.77 (step 650k)**
Episode length moyenne	Souvent 624 (max), avec dips ponctuels	Similaire: souvent 624, avec dips	Similaire: souvent 624, avec dips
Policy loss finale	0.027 -> 0.045 (stable)	0.024 (stable)	0.025-0.027 (stable)

Critere	E1	E2	E3
Value loss finale	0.007 -> 1.10-1.25 (instabilite legere)	0.004 -> 1.47 (instabilite plus marquee)	0.005 -> 1.22 (meilleure stabilite!)
Taux de succes inference	1/2 (50%, 1 crash, 1 tour 23.26s)	~1/4 ou moins (25%, varios crashes, 1 tour 20.18s)	**Meilleur (~75%+), rare de crash, 1 tour 20.91s**
Temps tour moyen (reussi)	23.26 sec	20.18 sec (plus rapide)	**20.91 sec (bon compromis)**
Duree du run	46 min	52 min (1h)	**50 min**
Progression courbe	Bruitee, oscillations	Bruitee, oscillations	**Propre et fulgurante, peu d'oscillations**

Analyse

A commenter simplement

- Le batch size plus grand ou plus petit.
- L'effet du learning rate.
- La stabilite des courbes.
- La difference entre performance et vitesse de calcul.
- Le temps d'entrainement par experience.
- Le temps moyen pour finir le trajet en inference (5 essais conseilles).
- Le nombre de crashs sur 5 essais inference (metrique de robustesse).

Limites

- Un seul run par configuration.
- Alea dans l'environnement.
- Temps de calcul sur la machine.
- Influence possible du reward shaping.

Bilan court

- **Ce qui marche** : les trois configurations convergent vers une reward positive (baseline 54.51, E2 60.05, E3 55.77). L'augmentation du batch size (E1→E2) accelere l'apprentissage mais risque l'instabilite; E3 trouve un equilibre optimal.
- **Ce qui reste instable** : E2 montre une value loss plus elevee (1.47) et un taux de crash en inference plus eleve. E1 est stable mais lent. E3 reconcilie ces deux tendances.
- **Ce qu'on garde comme meilleure configuration** : **E3** offre le meilleur compromis : reward finale 55.77, value loss 1.22, robustesse en inference superieure, et progression propre sans oscillations excessives.

Conclusion

Synthese

Les trois experiences ont montre que l'ajustement du batch size et du learning rate ont un impact majeur sur le compromis stabilite/performance. E1 (baseline) offre une stabilite acceptable mais une convergence lente. E2 accelere l'apprentissage (reward max 60.05) mais au prix d'une instabilite accrue (value loss 1.47, crashes en inference). E3 reconcilie ces deux approches : avec un batch size intermediaire (768) et un learning rate modere (2.2e-4), E3 atteint une reward finale de 55.77, une value loss maitrisee a 1.22, et surtout une robustesse en inference nettement superieure aux deux autres configurations.

Configuration retenue

E3 - Configuration optimale pour le projet TP2 :

Parametre	Valeur	Justification
`batch_size`	768	Compromis entre E1 (512) et E2 (1024) pour lisser les gradients sans exces
`learning_rate`	2.2e-4	Entre E1 (3e-4) et E2 (1.5e-4) pour eviter variance tout en gardant progression rapide
`epsilon`	0.17	PPO clip modere : plus strict que E1 (0.2) mais moins rigide que E2 (0.15)

Parametre	Valeur	Justification
`beta`	0.004	Equilibre exploration/exploitation entre E1 (0.005) et E2 (0.003)
`lambda`	0.95	Inchange, GAE discount factor optimal
`num_epoch`	3	Inchange, suffisant pour convergence
`max_steps`	650000	Limite pratique choisie pour un run d'environ 50 min

Resultat E3 : Reward 55.77 (step 650k), Value Loss 1.22, temps tour 20.91s, stabilite en inference ~75%+ de succes.

Amelioration future possible

Une continuation d'E3 pour ~100k steps supplementaires (aurai pris ~15 min) aurait probablement permis d'atteindre 57-58 en reward sans perdre la robustesse observee; sinon, une strategie de **learning rate decay** (diminution progressive du LR) pourrait reduire davantage la variance de la value loss en fin d'apprentissage.

Annexe

Lancement

```
mlagents-learn results/configuration_example.yaml --run-id=Experience1 --force
```

Rappel

Dans la console, chaque run cree son propre dossier de resultats, par exemple `results/Experience1` .

TensorBoard

```
tensorboard --logdir "C:\Users\Ajax\AI_in_games_course\results" --port 6006
```

Test du modele dans Unity

- Utiliser le fichier modele `.onnx` du run (par exemple `results/Experience1/car_agent.onnx`).

- Le glisser dans le champ `Model` des Behavior Parameters de l'agent.